

Testing & Evaluation Report

1. System Overview

This project builds a simple student-support ticket classifier. Given a short ticket text, the system predicts one of five categories (IT, Fees, Timetable, Exams, General) and assigns a priority (Low/Medium/High). For each prediction it also returns a confidence score, and the pipeline produces basic monitoring outputs (overall metrics, daily counts, and a light drift signal based on confidence over time).

2. Architecture Choice and Justification

Chosen style: layered, single-codebase Python system, with an event-queue simulation.

- Maintainability. The work is split into clear modules (data generation, training, inference, event bus, monitoring). Each part has a narrow job, so changes don't spread everywhere.
- Scalability. The queue mimics an event-driven flow: inference emits events; monitoring consumes them. In a real deployment, you could add more consumers without touching inference.
- Reliability. Monitoring is not on the critical path. If monitoring fails, events can still be buffered and processed later.
- Basic security. The interface is simple (text in → structured event out). That makes validation, logging, and boundary checks easier.

3. SDLC Stages Followed

1. Planning. I kept the scope small on purpose: 5 ticket categories + 3 priority levels. "Done" meant an end-to-end run that prints metrics and monitoring outputs.
2. Requirements. I wrote down what the system must produce (predicted label + priority + confidence, plus a confusion matrix and a simple drift signal) and what pieces are needed to do that (data generator, trainer, inference, event queue, monitoring).
3. Design. Before coding, I fixed two simple formats: (a) the dataset columns, and (b) the event message schema that gets pushed through the queue. I then used a layered layout with a queue simulation to keep modules separated.
4. Implementation. Modules were built one by one, then

connected into a single runnable pipeline. I avoided “hidden” steps outside the pipeline so the run is repeatable.

5. Testing. I first sanity-checked training and inference on small samples, then ran the full pipeline and verified that monitoring numbers update correctly (counts, metrics, confidence trend).

6. Deployment (conceptual). The trained model and vectorizer are saved as files (model.pkl / vectorizer.pkl) so inference can load them without retraining.

7. Evolution. If this moves beyond a demo, the obvious upgrades are: more realistic ticket text, tighter labels, and a drift check that triggers alerts instead of just printing averages.

4. Baseline Evaluation Results

A synthetic dataset of 300 tickets was used for training/validation. The baseline model is TF-IDF + Logistic Regression. Monitoring outputs:

- Accuracy: 1.0
- Macro Precision: 1.0
- Macro Recall: 1.0
- Macro F1: 1.0

The confusion matrix (true rows, predicted columns) shows perfect separation on this synthetic set.

Daily high-priority counts were also computed, for example:

- 2026-02-10: 6
- 2026-02-11: 13
- 2026-02-12: 16
- 2026-02-13: 23
- 2026-02-14: 24
- 2026-02-15: 11
- 2026-02-16: 19
- 2026-02-17: 14

The example range is roughly 0.80–0.83 over the observed dates.

Discussion: The perfect scores are not surprising: the data are synthetic and the text patterns are strongly tied to each class. This is useful as a pipeline check, but real tickets will be messier (mixed intent, vague wording, typos), so performance is expected to drop.

5. Failure Cases

I tested a couple of “messy” tickets on purpose, because real tickets rarely stay inside one clean category.

Failure Case 1:

Text: “I can’t access Moodle and my fee payment failed at the same time.”

Expected category: Fees

Predicted category: Fees

Priority: High

Confidence: 0.476

Analysis:

Although the prediction matches the expected category, the confidence is relatively low because the ticket contains two intents (“access Moodle” → IT, “fee payment failed” → Fees). With TF-IDF features, keywords from both intents compete, reducing the margin between the top class and other plausible classes. In a real system, low-confidence mixed-intent tickets should be flagged for manual review or routed to a “multi-label / needs clarification” path.

Failure Case 2

Text: “My exam timetable is missing in Moodle.”

Expected category: Exams

Predicted category: Timetable

Priority: Low

Confidence: 0.523

Analysis:

This is a true misclassification. The sentence includes strong timetable-related terms (“timetable”, “missing”), which likely dominate the TF-IDF representation, while the word “exam” is not strong enough to shift the decision boundary toward the Exams class. This illustrates a limitation of a simple TF-IDF + Logistic Regression baseline when handling overlapping vocabulary between categories (e.g., “timetable” appears in both lecture schedules and exam schedules).

A practical fix would be to increase training diversity with more “exam timetable” examples, add simple keyword/phrase rules (e.g., if “exam timetable” appears, boost Exams), or move to a model better at capturing context (e.g., fine-tuned transformer or sentence embeddings).

6. Improvement Ideas

Next steps that would make the system closer to real use:

- More realistic data. Add varied phrasing, typos, and multi-intent tickets.
- Better priority handling. Refine rules or train a separate priority classifier.
- Model/feature tuning. Try n-grams, different regularisation, or stronger models.
- Drift alerts. Add thresholds when confidence trends drop over a window.
- Human-in-the-loop. Route low-confidence tickets to manual review.